

AhamX Bodhi

Where I Becomes Infinite

Level 3: Applications

Module 2: Generative AI – Core Concepts & Architecture
Prof. Sashikumar Ganesan · IISc Bengaluru



Session Overview

Session Details

- **Module:** 2 – Core Concepts & Architecture
- **Duration:** 10 minutes
- **Bloom's Level:** B3 (Apply)
- **Difficulty:** L2 (Intermediate)
- **Sequence:** Concept 3 of Module 2

The Central Question

A foundation model adapted to a task is still just a model. It generates tokens when called.

How does it become a product someone uses?

This session introduces the architecture patterns that turn language models into reasoning agents, conversational assistants, and autonomous systems.



Learning Objectives

By the End of This Session You Will Be Able To

1. **Define** what an AI application is and how it differs from a raw model
2. **Describe** the anatomy of an AI agent: perception, reasoning, memory, and action
3. **Explain** the ReAct (Reasoning + Acting) loop and why it enables multi-step tasks
4. **Identify** tool use, function calling, and multi-agent coordination as key patterns
5. **Map** real production systems – chatbots, coding assistants, research agents – to their underlying architecture



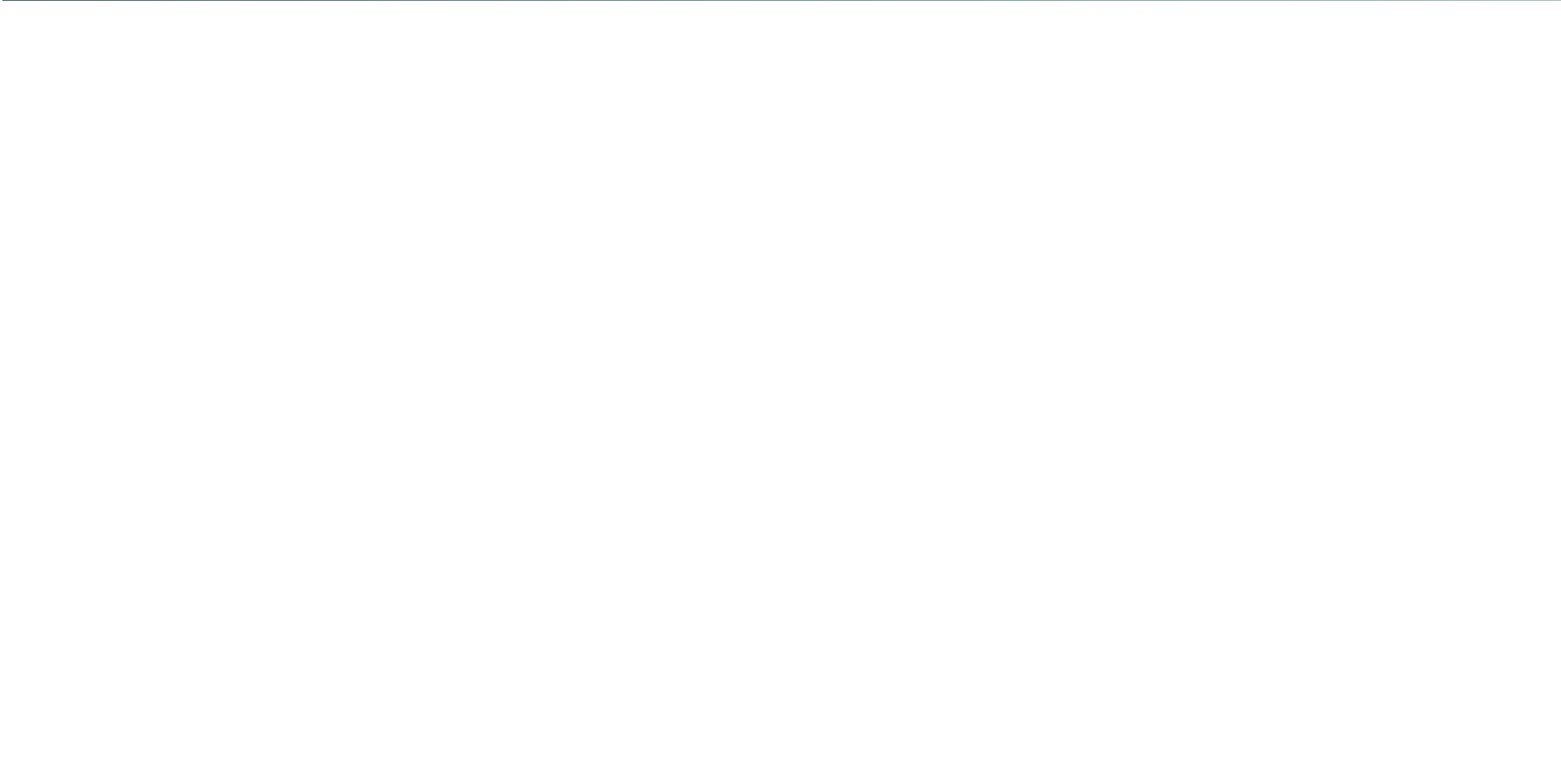
Prerequisites and Context

You Should Already Know

- Foundation models and self-supervised pre-training
- The three adaptation strategies: prompting, RAG, fine-tuning
- How a language model generates tokens autoregressively
- The Five-Level GenAI Framework (Level 3 = Build Applications)

This Session Unlocks

- Agentic AI Foundations (Module 5)
- ReAct Framework deep-dive (Module 5)
- Model Context Protocol – MCP (Module 5)
- Multi-agent system design (Module 9)
- Production LLMOps and safety (Module 9)





What Makes an AI Application?

The Missing Layer

A foundation model takes tokens in and produces tokens out. An **application** adds the surrounding structure that makes this useful: it decides *when* to call the model, *what context* to supply, *what to do with the output*, and *how to respond* to the user or environment. This surrounding structure is the application layer.

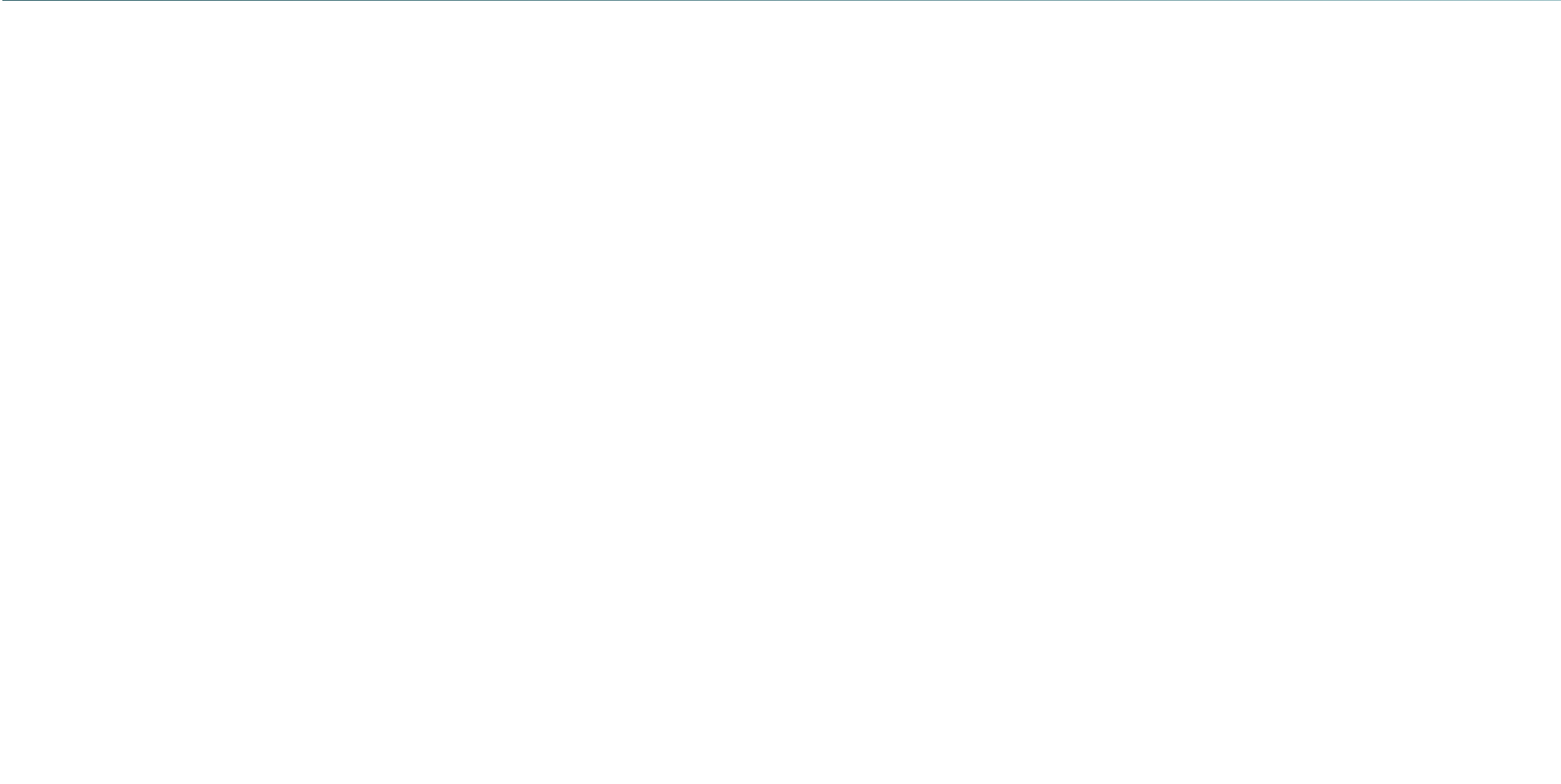
The Four Components of Every AI Application

- **Input handling:** Accepting user queries, documents, images, or API events
- **Context assembly:** Constructing the prompt – system prompt, retrieved chunks, history
- **Model invocation:** Calling the LLM (local or API) with the assembled context
- **Output handling:** Parsing the response, executing tool calls, formatting for display

Taxonomy of GenAI Applications



Application Type	Description	Key Pattern	Examples
Chatbot / Assistant	Multi-turn dialogue over a topic or document set	Prompt + memory	ChatGPT, Claude.ai, Gemini
RAG Application	Grounded Q&A from a private document corpus	Retrieve + generate	Enterprise search, Harvey AI
Code Assistant	Autocomplete, review, and generation for software	Context + completion	Copilot, Cursor, Claude Code
Tool-Using Agent	LLM calls external tools to complete tasks	ReAct loop	OpenAI Assistants, Devin
Multi-Agent System	Multiple specialised agents collaborate	Orchestrator + workers	LangGraph, AutoGPT, Crew AI
Multimodal Application	Processes text, images, audio, and video together	Multi-encoder fusion	GPT-4o, Gemini 1.5, Claude 3.5





What Is an AI Agent?

Definition

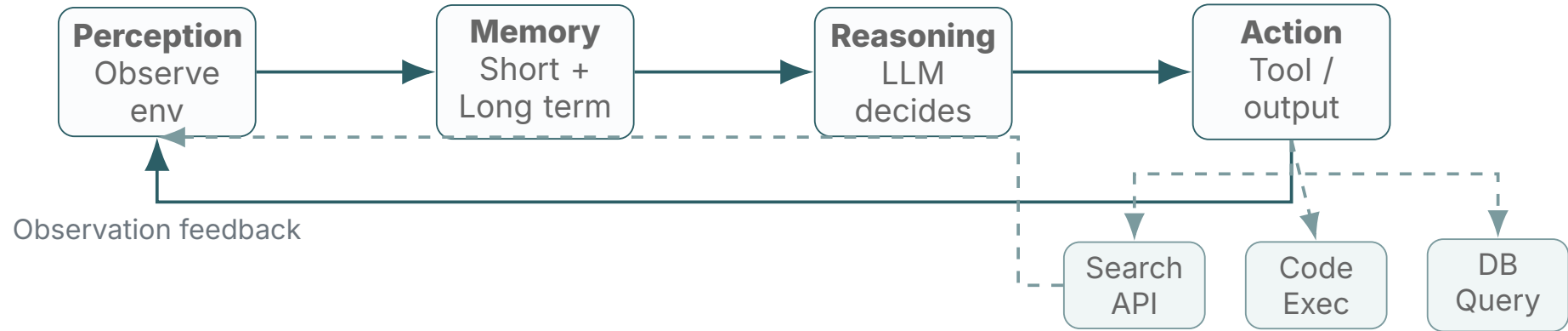
An **AI agent** is a system that perceives its environment, reasons about a goal, selects actions, executes those actions through tools or APIs, observes the results, and iterates until the goal is achieved or a stopping condition is met. The foundation model provides the reasoning; the agent loop provides the structure for sustained, multi-step behaviour.

The Four Components of Agent Anatomy

- **Perception:** Receiving observations – user input, tool outputs, environment state
- **Memory:** Storing and retrieving context – in-context (short-term) and external (long-term)
- **Reasoning:** The LLM interprets observations, decides the next action, generates a plan
- **Action:** Executing tool calls, writing files, calling APIs, generating final output



Agent Anatomy: The Perception--Reason--Act Loop



The loop is **iterative**: each action produces an observation that feeds back into perception, updating the agent's context for the next reasoning step.

Memory in AI Agents



In-Context Memory (Short-Term)

- Everything in the current context window
- Conversation history, tool outputs, instructions
- Bounded by context length (4K–2M tokens)
- Lost when the session ends
- Fast: no retrieval latency

External Memory (Long-Term)

- Stored outside the model in a vector database or key-value store
- Retrieved by semantic similarity at each reasoning step
- Persistent across sessions; scales to millions of entries
- Implemented with Pinecone, Weaviate, Redis, or Postgres
- Enables agents to “remember” users and past tasks





ReAct: Reasoning + Acting

What Is ReAct?

ReAct (Yao et al., 2022) is an agent prompting framework that interleaves **reasoning traces** (Chain-of-Thought) with **action steps** (tool calls) in a single context window. The model alternates between thinking through the problem and acting on that reasoning, making its decision process transparent and correctable.

The ReAct Loop – Four Steps Per Iteration

- **Thought:** The model reasons aloud – “I need to find the population of Mumbai...”
- **Action:** The model issues a structured tool call – `search('Mumbai population 2024')`
- **Observation:** The tool returns a result – “Mumbai: 20.7 million (2024 census)”
- **Repeat or Answer:** The model decides whether more steps are needed or generates the final answer



ReAct Trace: Worked Example

Thought 1: I need today's Nifty 50 close price. I should search for it.



Action 1: `web_search('Nifty 50 close today')`



Observation 1: Nifty 50 closed at 22,474 on Feb 23, 2026.



Thought 2: I have the price. Now I can compare with yesterday's close of 22,310.



Action 2: Calculate: $(22474 - 22310) / 22310 \times 100 = 0.74\%$



Answer: The Nifty 50 rose by approximately **0.74%** today.

Tool Use and Function Calling

The right's choice is —

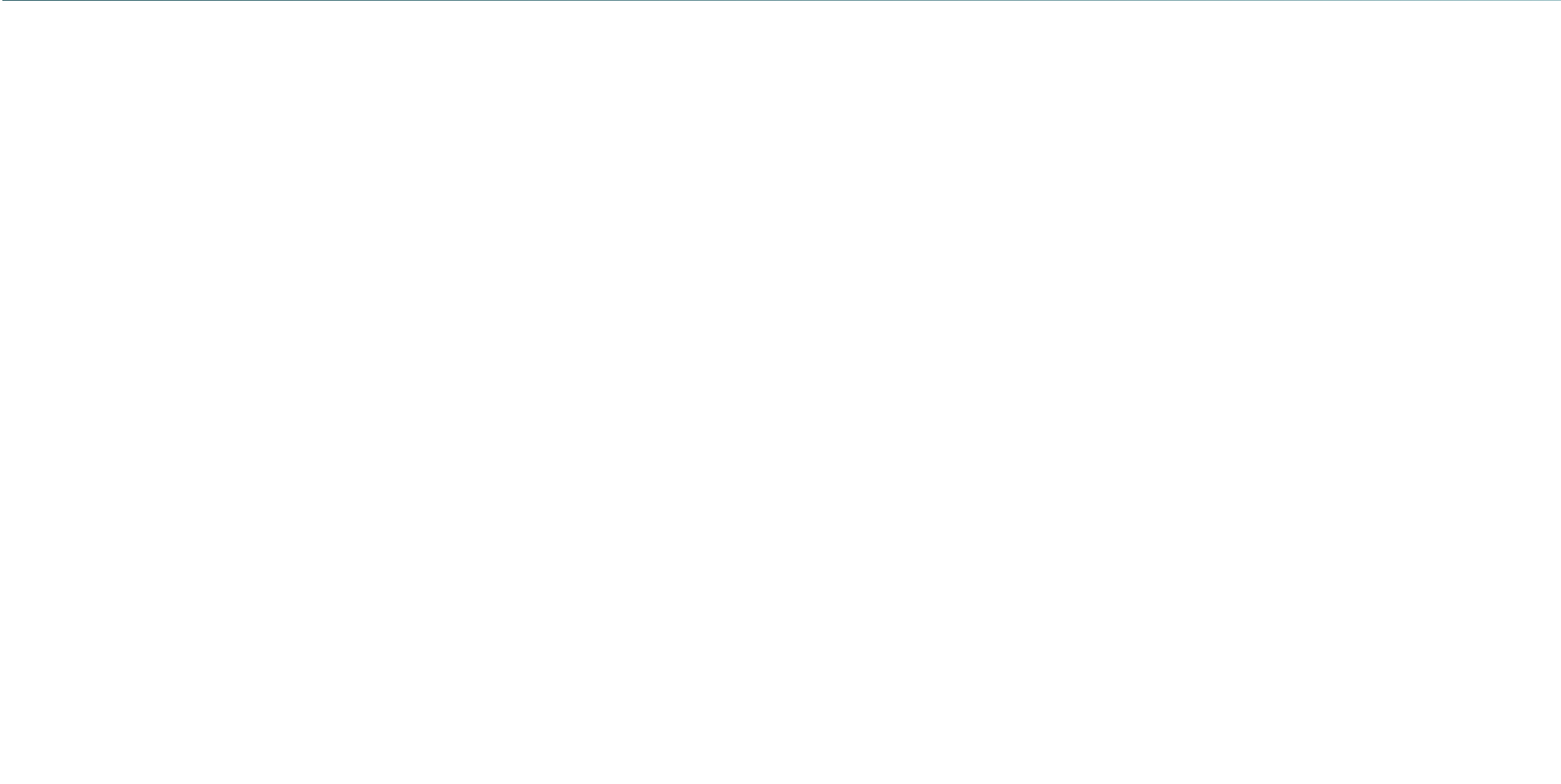


How Agents Access External Capabilities

Modern LLM APIs support **function calling** (OpenAI) and **tool use** (Anthropic): the developer provides a JSON schema of available functions, and the model returns structured JSON specifying which function to call and with what arguments. The application executes the call, then feeds the result back as a new message. **No special fine-tuning is required** – the model learns this from its training.

Common Tools in Production Agents

- **Web search:** Perplexity API, Tavily, Serper, Google Search API
- **Code execution:** OpenAI Code Interpreter, E2B sandbox, Jupyter kernels
- **Database queries:** SQL over relational DBs, vector similarity search
- **File operations:** Read, write, and process PDFs, spreadsheets, images
- **API calls:** Weather, calendar, payment, CRM, and any REST endpoint



Multi-Agent Systems



Why Multiple Agents?

A single agent faces bottlenecks: context window limits prevent handling very long tasks, and a generalist model may underperform a specialist. **Multi-agent systems** decompose complex goals into sub-tasks, assign them to specialised agents, and coordinate results through an **orchestrator**. Each agent has its own context, tools, and model (potentially different base models).

Multi-Agent Systems



Key Multi-Agent Patterns

- **Orchestrator-Worker:** A planner agent decomposes the task; worker agents execute sub-tasks
- **Debate / Critique:** One agent proposes, a second critiques, producing higher-quality outputs
- **Parallel Execution:** Independent sub-tasks run concurrently, reducing total latency
- **Specialist Routing:** An orchestrator routes queries to domain specialists (legal, medical, coding)
- **Hierarchical:** Nested orchestrators manage complex, long-horizon plans



Model Context Protocol (MCP)

What Is MCP?

- An open standard (Anthropic, 2024) for connecting AI models to external tools and data sources
- Defines a protocol for **servers** (tools/data) and **clients** (models) to communicate
- Replaces ad-hoc tool integration with a uniform interface
- Enables any MCP-compatible model to use any MCP-compatible tool without custom glue code

Why MCP Matters

- **Interoperability:** Tools built for Claude work with any MCP-compliant model
- **Ecosystem:** Growing library of MCP servers – GitHub, Slack, Google Drive, Postgres
- **Security:** Standardised permission model for tool access
- **Composability:** Agents can discover and chain MCP tools dynamically at runtime





Application: Conversational AI Assistants

Architecture

- System prompt sets persona, constraints, knowledge boundaries
- Conversation history maintained in-context
- Optional RAG layer for knowledge grounding
- Memory store for persistent user preferences
- Rate limiting and safety guardrails at the API layer

Production Systems

- **ChatGPT:** GPT-4o backbone; 100M+ daily users; memory-enabled
- **Claude.ai:** 200K token context; Projects for persistent memory
- **Gemini:** Deep Google Search integration; 1M token context
- **Copilot (Microsoft):** Embedded in Teams, Word, Excel, Outlook
- **Meta AI:** Llama-based; integrated in WhatsApp, Instagram



Application: AI Coding Agents

Beyond Autocomplete

Modern coding agents are not passive autocomplete systems. They operate as **agentic loops**: reading the entire codebase via RAG, writing and running tests, reading error output as observations, and iterating until tests pass. The ReAct framework is the core architecture: think, act (write/run code), observe (test output), and repeat. This loop can run for dozens of iterations on a single complex task.

Production Examples

- **GitHub Copilot Workspace**: Full-task agent from issue to pull request
- **Cursor Composer**: Multi-file editing with repo-level RAG context
- **Claude Code (Anthropic)**: Terminal-based agentic coding with MCP tools
- **Devin (Cognition)**: Autonomous software engineer with shell, browser, editor tools
- **SWE-agent**: Open-source benchmark on SWE-bench – solves real GitHub issues



Application: Research and Data Analysis Agents

Research Agents

- Query academic databases (PubMed, Semantic Scholar, arXiv)
- Read and synthesise full-text papers from PDF
- Generate structured summaries and citation lists
- Identify contradictions and research gaps automatically
- Example: Elicit.com – AI research assistant for systematic reviews

Data Analysis Agents

- Accept CSV/Excel upload; generate and execute analysis code
- Produce visualisations, statistical summaries, and narrative interpretation
- Iterate on analysis based on follow-up questions
- Examples: ChatGPT Advanced Data Analysis, Julius AI, Claude Artifacts
- Underlying pattern: tool use (Python execution) + RAG (data context)



Application: Intelligent Customer Support Systems

Multi-Agent Customer Support Architecture

Enterprise customer support illustrates the full multi-agent pattern. An **orchestrator agent** classifies the intent and routes to specialist agents: a **knowledge agent** answers FAQs via RAG; an **action agent** queries order databases and processes returns; an **escalation agent** prepares summaries for human handoff when confidence is low. All agents share a common conversation context but maintain separate tool registries.

Production Deployments

- **Intercom Fin:** GPT-4-powered; resolves 50%+ of tickets without human
- **Salesforce Einstein:** Multi-agent CRM integration with Slack and email
- **ServiceNow Now Assist:** IT service management agent with tool calling
- **Zendesk AI:** Auto-triage, draft replies, and macro suggestions



Application: Scientific AI Agents

Drug Discovery Agents

- Orchestrate across protein structure prediction (AlphaFold 3), molecular simulation, and literature search
- Propose candidate molecules, predict ADMET properties, rank by drug-likeness
- Insilico Medicine: generative agent designed two clinical candidates in 18 months
- Pattern: multi-agent with specialised domain tools

Climate and Physics Agents

- Orchestrate climate models, observation databases, and simulation tools
- Autonomously design and run experiments to test hypotheses
- FutureHouse: AI co-scientist platform for biology and chemistry
- Google DeepMind AlphaGeometry: autonomous geometry theorem proving
- Pattern: long-horizon planning + specialised scientific tool libraries



Knowledge Graph: Where This Concept Lives

Prerequisite Concepts

- Level 1: Foundation Models (Module 2, Concept 1)
- Level 2: Model Adaptation (Module 2, Concept 2)
- What is a Language Model? (Module 1)
- Generative vs. Discriminative Models (Module 1)

Concepts This Unlocks

- What Is an AI Agent? (Module 5)
- ReAct Framework deep-dive (Module 5)
- Model Context Protocol (Module 5)
- Multi-Agent System Architecture (Module 9)
- Production Safety Guardrails (Module 9)

Common Misconceptions



Critical Corrections

- **"A chatbot is an AI agent"** – A chatbot responds to a single turn; an agent maintains a goal-directed loop across multiple steps and actions. The difference is the agentic loop, not the interface
- **"Agents understand the task"** – Agents simulate purposeful behaviour through next-token prediction conditioned on the goal description; there is no deep comprehension
- **"More agents always give better results"** – Multi-agent systems amplify both strengths and errors; a well-designed single-agent system often outperforms a poorly coordinated multi-agent one
- **"Tool calling requires fine-tuning"** – Modern LLMs (GPT-4, Claude 3.5, Llama 3) support structured tool use natively through instruction tuning done by the model provider
- **"Long context window replaces memory systems"** – Long context is expensive, slow, and not selective; external memory retrieval is cheaper and targets only relevant information

Session Summary

Key Takeaways

- Applications wrap models with input handling, context assembly, and output routing
- AI agents add a perception-memory-reasoning-action loop for multi-step goals
- ReAct interleaves thought and action traces, making agent reasoning transparent
- Tool use and MCP extend agents beyond language into real-world action
- Multi-agent patterns handle complexity that exceeds a single agent's context or capability

What Comes Next

- **In Module 2:** Level 4 – Integration (scaling agents to production)
- **Module 5:** Build your first ReAct agent with LangChain and real tools
- **Module 9:** Multi-agent orchestration with LangGraph and production safety
- **Lab prep:** Install LangChain and obtain an OpenAI or Anthropic API key
- Every capstone project builds on the application patterns introduced here



Thank You

Module 2: Generative AI – Core Concepts & Architecture
Level 3: Applications · Prof. Sashikumaar Ganesan · IISc
Bengaluru